

Playlist Embedding Analysis

This notebook explores a dataset of one million Spotify playlists represented as dense vector embeddings. Each playlist has been encoded into a 768-dimensional vector using a pre-trained language model (specifically, a BERT-style model with mean pooling over the token embeddings of the playlist's track names). The resulting embeddings capture semantic relationships between playlists — playlists with similar musical themes or track overlap should appear close together in embedding space.

The dataset is stored as a serialized Python dictionary (`.pk1` file) shared via Google Drive. It contains three parallel sub-dictionaries, each indexed by playlist ID:

- `playlist_embeddings`: the 768-dimensional mean-pooled embedding for each playlist
- `playlist_titles`: the human-given name of each playlist
- `playlist_tracks`: the list of track names included in each playlist

The analysis proceeds in three parts: (1) summary statistics describing the dataset at a high level, (2) manual inspection of individual playlists to build intuition, and (3) a nearest-neighbor search and UMAP visualization to explore the geometry of the embedding space.

Setup: Mounting Google Drive

The dataset is stored in Google Drive and accessed via a shared shortcut added to the Colab Notebooks folder. The cell below mounts Drive so the file system is accessible from within this Colab session. The `force_remount=True` argument ensures a clean connection even if Drive was previously mounted.

```
# Mount Google Drive so Colab can access files stored there
# force_remount=True ensures a fresh connection if Drive was already mounted
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

Mounted at /content/drive

Locating the Dataset

Because the `.pk1` file was originally shared externally (not owned by this Google account), it does not appear directly in `MyDrive`. To make it accessible, a Drive shortcut was created pointing to the shared file and placed in the Colab Notebooks folder. The cell below walks the entire mounted Drive file system and prints the resolved path of any `.pk1` files it finds, confirming that the shortcut is accessible.

```
import os

# Walk through all folders in Google Drive to find any .pk1 files
# The .pk1 file was shared externally, so a shortcut was added to Colab Notebooks
# to make it accessible here
for root, dirs, files in os.walk('/content/drive'):
    for file in files:
        if file.endswith('.pk1'):
            print(os.path.join(root, file))

/content/drive/.shortcut-targets-by-id/1uAlhCndAH2C0kwEWSL-nWp601P305CX6/calced_embeddings/mean_pooled_embeddings.pk1
```

Loading the Dataset

The dataset is stored in Python's pickle format, which serializes arbitrary Python objects to disk. We use the built-in `pickle` module to deserialize the file back into memory. The result is a dictionary with three keys, each mapping to a sub-dictionary of one million entries. We confirm the structure by printing the type and number of top-level keys.

```
import pickle

# Full path to the .pk1 file (resolved through the Drive shortcut)
path = '/content/drive/.shortcut-targets-by-id/1uAlhCndAH2C0kwEWSL-nWp601P305CX6/calced_embeddings/mean_pooled_embeddings.pk1'

# Open and deserialize the .pk1 file into a Python object
with open(path, 'rb') as f:
    data = pickle.load(f)

# Confirm it loaded as a dict and check how many top-level keys it has
print(type(data))
print(data.shape if hasattr(data, 'shape') else len(data))
```

```
<class 'dict'>
3
```

Inspecting the Top-Level Structure

The loaded object is a dictionary with three keys. Each value is itself a dictionary mapping playlist IDs to data. The cell below iterates over the top-level keys and prints the name, type, and size of each sub-dictionary, confirming that all three contain entries for the same 1,000,001 playlist IDs (including one header row).

```
# Inspect each of the 3 top-level keys in the data dict
# Shows the key name, its type, and either its shape (if array) or length (if dict/list)
for key, value in data.items():
    print(key, type(value), value.shape if hasattr(value, 'shape') else len(value))

playlist_embeddings <class 'dict'> 1000001
playlist_titles <class 'dict'> 1000001
playlist_tracks <class 'dict'> 1000001
```

Identifying the Header Row

The dataset contains 1,000,001 entries rather than exactly 1,000,000. Inspecting the very first key reveals that the first entry is a header row with the key `'pid'` and placeholder values (`'name'`, `['track_name']`), rather than real playlist data. This entry was likely inherited from the original CSV source file. It must be excluded from all downstream analysis. We also confirm the embedding dimensionality here: each real playlist is represented as a 768-dimensional vector.

```
# Peek at the very first entry in each sub-dict
# Note: this returns the header row (key='pid'), not real playlist data
sample_key = next(iter(data['playlist_embeddings']))

print("Key:", sample_key)
print("Title:", data['playlist_titles'][sample_key])
print("Tracks:", data['playlist_tracks'][sample_key])
print("Embedding shape:", data['playlist_embeddings'][sample_key].shape)

Key: pid
Title: name
Tracks: ['track_name']
Embedding shape: (768,)
```

Verifying a Real Playlist Entry

Skipping the header row, we examine the second entry in the dataset — the first real playlist. We use `itertools.islice` to efficiently access a specific position in the dictionary without converting the entire key set to a list. This confirms the expected data structure: each playlist has an integer ID, a human-readable title, a list of track names, and a 768-dimensional embedding vector.

```
import itertools

# Skip the first entry (header row) and grab the second entry, which is real playlist data
# Each playlist has a 768-dimensional embedding vector (from a BERT-style language model)
sample_key = next(itertools.islice(data['playlist_embeddings'], 1, 2))

print("Key:", sample_key)
print("Title:", data['playlist_titles'][sample_key])
print("Tracks:", data['playlist_tracks'][sample_key][:5]) # first 5 tracks
print("Embedding shape:", data['playlist_embeddings'][sample_key].shape)

Key: 0
Title: Throwbacks
Tracks: ['Lose Control (feat. Ciara & Fat Man Scoop)', 'Toxic', 'Crazy In Love', 'Rock Your Body', "It Wasn't Me"]
Embedding shape: (768,)
```

Convenience References

For cleaner code throughout the rest of the notebook, we extract the three sub-dictionaries into their own named variables and filter out the header row. All subsequent analysis operates on `real_keys`, which contains only the integer playlist IDs corresponding to actual playlists.

```
# Shorthand references to each sub-dict for convenience
embeddings = data['playlist_embeddings']
```

```

titles = data['playlist_titles']
tracks = data['playlist_tracks']

# Filter out the header row – real playlist keys are integers
real_keys = [k for k in titles.keys() if k != 'pid']

# Confirm the count and embedding type for one entry
print(f"Total real playlists: {len(real_keys):,}")
first_key = real_keys[0]
print("Example key:", first_key)
print("Type of embedding value:", type(embeddings[first_key]))
print("Embedding shape:", embeddings[first_key].shape)

```

```

Total real playlists: 1,000,000
Example key: 0
Type of embedding value: <class 'numpy.ndarray'>
Embedding shape: (768,)

```

1. Summary Statistics

Before analyzing the embedding space, we characterize the dataset at a high level. This includes the total number of playlists, the distribution of playlist title lengths, the most frequently occurring playlist titles, and the distribution of track counts per playlist. These statistics provide a sense of the scale and diversity of the dataset and can reveal patterns such as whether users tend to give playlists short names or whether playlist length is roughly uniform.

```

import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# This code chunk does not make all entries lowercase before analyzing.
# This code chunk may be deleted because import statements are repeated in a subsequent chunk.

print(f"Total playlists: {len(real_keys):,}")

# --- Title length statistics ---
title_lengths = [len(titles[k]) for k in real_keys] # character length of each title
print(f"\nPlaylist title lengths (characters):")
print(f"  Mean:    {np.mean(title_lengths):.1f}")
print(f"  Median:  {np.median(title_lengths):.1f}")
print(f"  Max:     {np.max(title_lengths)}")
print(f"  Min:     {np.min(title_lengths)}")

# Most common playlist titles across the dataset
title_counts = Counter(titles[k] for k in real_keys)
print(f"\nTop 10 most common playlist titles:")
for title, count in title_counts.most_common(10):
    print(f"  '{title}': {count:,}")

# --- Track count statistics ---
track_counts = [len(tracks[k]) for k in real_keys] # number of songs per playlist
print(f"\nTracks per playlist:")
print(f"  Mean:    {np.mean(track_counts):.1f}")
print(f"  Median:  {np.median(track_counts):.1f}")
print(f"  Max:     {np.max(track_counts)}")
print(f"  Min:     {np.min(track_counts)}")

```

```
Total playlists: 1,000,000
```

```

Playlist title lengths (characters):
  Mean:    7.7
  Median:  7.0
  Max:     100
  Min:     1

```

```
Top 10 most common playlist titles:
```

```

  'Country': 5,388
  'Chill': 4,945
  'Rap': 4,793
  'Christmas': 4,779
  'Rock': 4,492
  'Workout': 4,434
  'Oldies': 4,262
  'chill': 4,070
  'country': 3,158
  'Worship': 3,076

```

```
Tracks per playlist:
```

```

  Mean:    66.3
  Median:  49.0
  Max:     376
  Min:     5

```

The total number of playlists is 1,000,000, a sample from Spotify's full dataset. The title lengths have a mean of 7.7 and a median of 7.0 characters. Most playlist titles appear to be short, even single words. The max of 100 characters is likely due to some people writing full sentences as a playlist title, but this is not common.

The most common playlist titles are "Country" and "Chill", however a lowercase "chill" and "country" are currently counted as different titles. We need to make sure that we can group those together, so all strings will be changed to lowercase so that they can be compared.

```
# Normalize all playlist titles to lowercase to merge case variants (e.g. 'Chill' and 'chill')
titles_lower = {k: v.lower() for k, v in titles.items() if k != 'pid'}

# Confirm a before/after example
example_key = real_keys[0]
print(f"Original: '{titles[example_key]}' -> Normalized: '{titles_lower[example_key]}'")
```

```
Original: 'Throwbacks' -> Normalized: 'throwbacks'
```

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

print(f"Total playlists: {len(real_keys):,}")

# Title length statistics (using lowercased titles)
title_lengths = [len(titles_lower[k]) for k in real_keys]
print(f"\nPlaylist title lengths (characters):")
print(f" Mean: {np.mean(title_lengths):.1f}")
print(f" Median: {np.median(title_lengths):.1f}")
print(f" Max: {np.max(title_lengths)}")
print(f" Min: {np.min(title_lengths)}")

# Most common titles – case variants now merged
title_counts = Counter(titles_lower[k] for k in real_keys)
print(f"\nTop 10 most common playlist titles (lowercased):")
for title, count in title_counts.most_common(10):
    print(f" '{title}': {count:,}")

# Track count statistics
track_counts = [len(tracks[k]) for k in real_keys]
print(f"\nTracks per playlist:")
print(f" Mean: {np.mean(track_counts):.1f}")
print(f" Median: {np.median(track_counts):.1f}")
print(f" Max: {np.max(track_counts)}")
print(f" Min: {np.min(track_counts)}")
```

```
Total playlists: 1,000,000
```

```
Playlist title lengths (characters):
```

```
Mean: 7.7
Median: 7.0
Max: 100
Min: 1
```

```
Top 10 most common playlist titles (lowercased):
```

```
'chill': 9,256
'country': 8,724
'rap': 8,045
'workout': 7,644
'oldies': 7,413
'christmas': 6,594
'rock': 6,351
'party': 5,487
'throwback': 5,141
'worship': 4,603
```

```
Tracks per playlist:
```

```
Mean: 66.3
Median: 49.0
Max: 376
Min: 5
```

The total count for the top playlist "chill" now includes all the playlist titles that had "chill" with any mix of capitalization.

The number of tracks per playlist has more variation than title. The median is significantly smaller than the mean, suggesting that there are some playlists with a large number of tracks pulling the mean up.

✓ Visualizing the Distributions

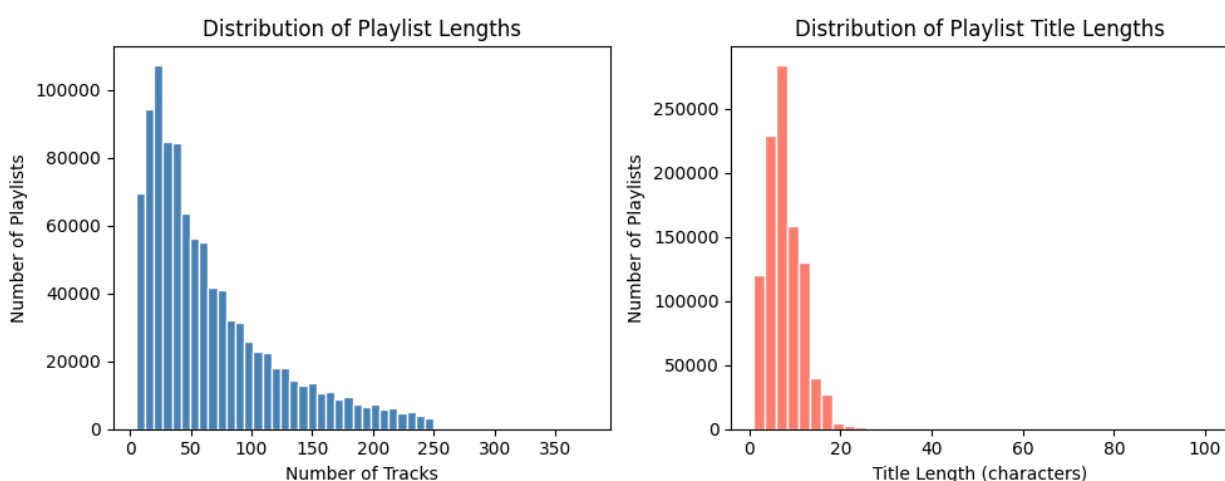
The histograms below show how playlist length (number of tracks) and title length (number of characters) are distributed across the full dataset. As expected from the mean and median, the playlist length distribution is right skewed suggesting that some playlists are much longer than the typical playlist. The title lengths appear to have a more symmetric normal distribution, however there is a small number of playlists with long titles.

```
# Plot distribution of track counts and title lengths side by side
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.hist(track_counts, bins=50, color='steelblue', edgecolor='white')
plt.xlabel('Number of Tracks')
plt.ylabel('Number of Playlists')
plt.title('Distribution of Playlist Lengths')

plt.subplot(1, 2, 2)
plt.hist(title_lengths, bins=40, color='salmon', edgecolor='white')
plt.xlabel('Title Length (characters)')
plt.ylabel('Number of Playlists')
plt.title('Distribution of Playlist Title Lengths')

plt.tight_layout()
plt.show()
```



Below is a Word Cloud of playlist titles. This visually shows the most common themes by increasing the size of the text with a higher frequency.

```
# !pip install wordcloud
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# Join all lowercased titles into one big string
all_titles = ' '.join(titles_lower[k] for k in real_keys)

wordcloud = WordCloud(width=1200, height=600, background_color='white',
                      colormap='viridis', max_words=100).generate(all_titles)

plt.figure(figsize=(14, 7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Most Common Words in Playlist Titles', fontsize=16)
plt.tight_layout()
plt.show()
```

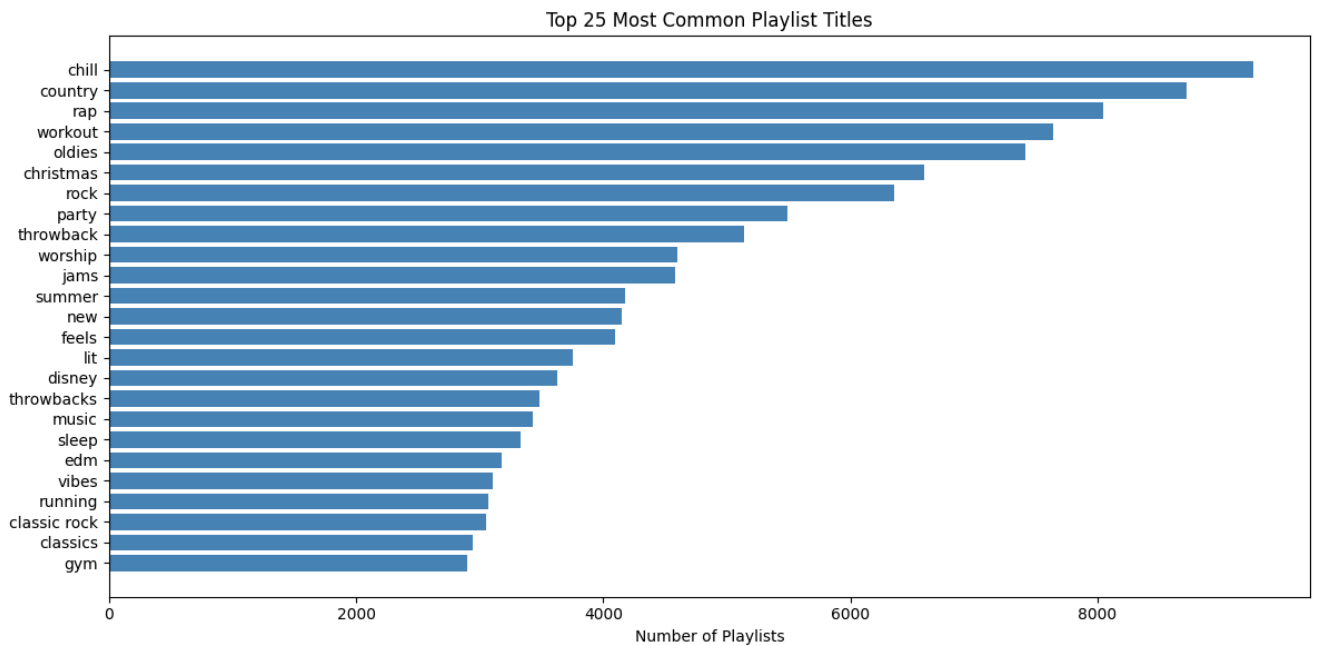


Bar chart showing the top 25 most common playlist titles and how many playlist contain that title.

```
import matplotlib.pyplot as plt
from collections import Counter

title_counts = Counter(titles_lower[k] for k in real_keys)
top_n = 25
labels, values = zip(*title_counts.most_common(top_n))

plt.figure(figsize=(12, 6))
bars = plt.barh(labels[::-1], values[::-1], color='steelblue')
plt.xlabel('Number of Playlists')
plt.title(f'Top {top_n} Most Common Playlist Titles')
plt.tight_layout()
plt.show()
```



Scatterplot of title length compared to track count. Do longer playlists have longer titles?

```
import matplotlib.pyplot as plt
import numpy as np

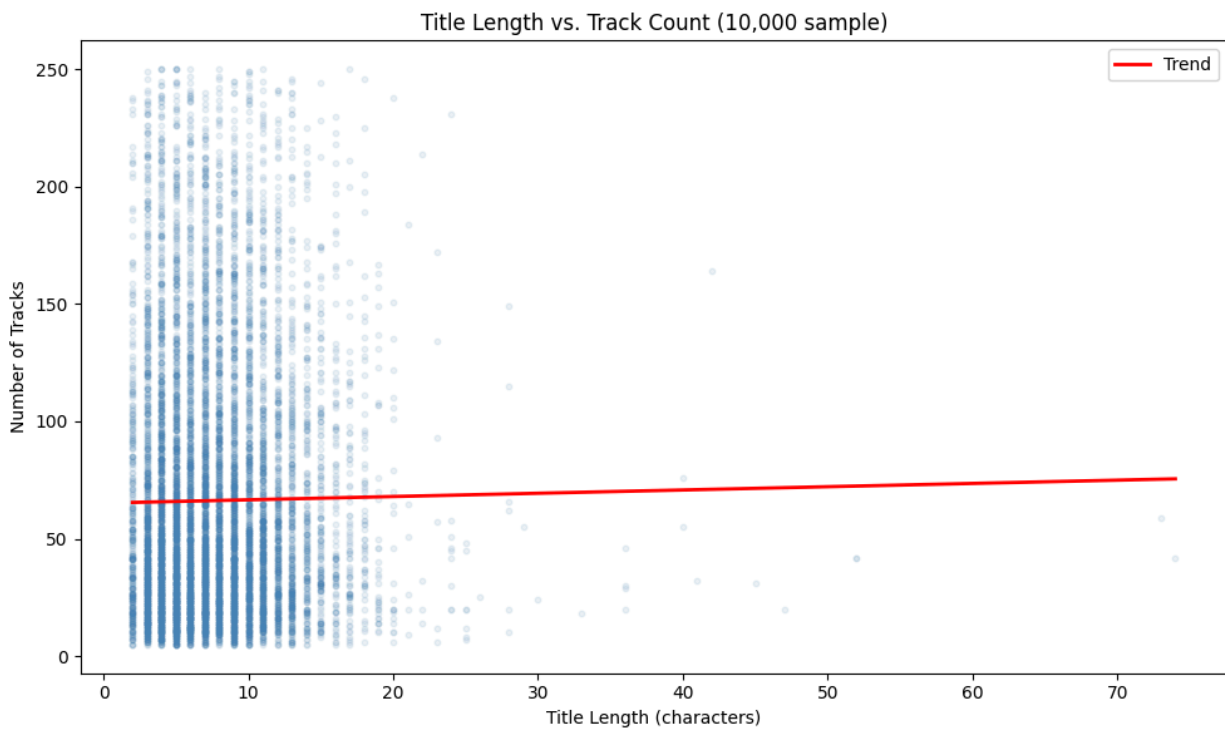
title_lengths = [len(titles_lower[k]) for k in real_keys]
track_counts = [len(tracks[k]) for k in real_keys]

# Sample 10000 points so the plot is not too dense
sample_size = 10000
idx = np.random.choice(len(real_keys), sample_size, replace=False)
x = [title_lengths[i] for i in idx]
y = [track_counts[i] for i in idx]

plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.1, s=10, color='steelblue')
plt.xlabel('Title Length (characters)')
plt.ylabel('Number of Tracks')
plt.title('Title Length vs. Track Count (10,000 sample)')

# Add a trend line
m, b = np.polyfit(x, y, 1)
x_line = np.linspace(min(x), max(x), 100)
plt.plot(x_line, m * x_line + b, color='red', linewidth=2, label='Trend')
plt.legend()
plt.tight_layout()
plt.show()

print(f"Trend line slope: {m:.2f} (tracks per additional character)")
```



Trend line slope: 0.14 (tracks per additional character)

CDF of track counts. What percentage of playlists have at most X tracks?

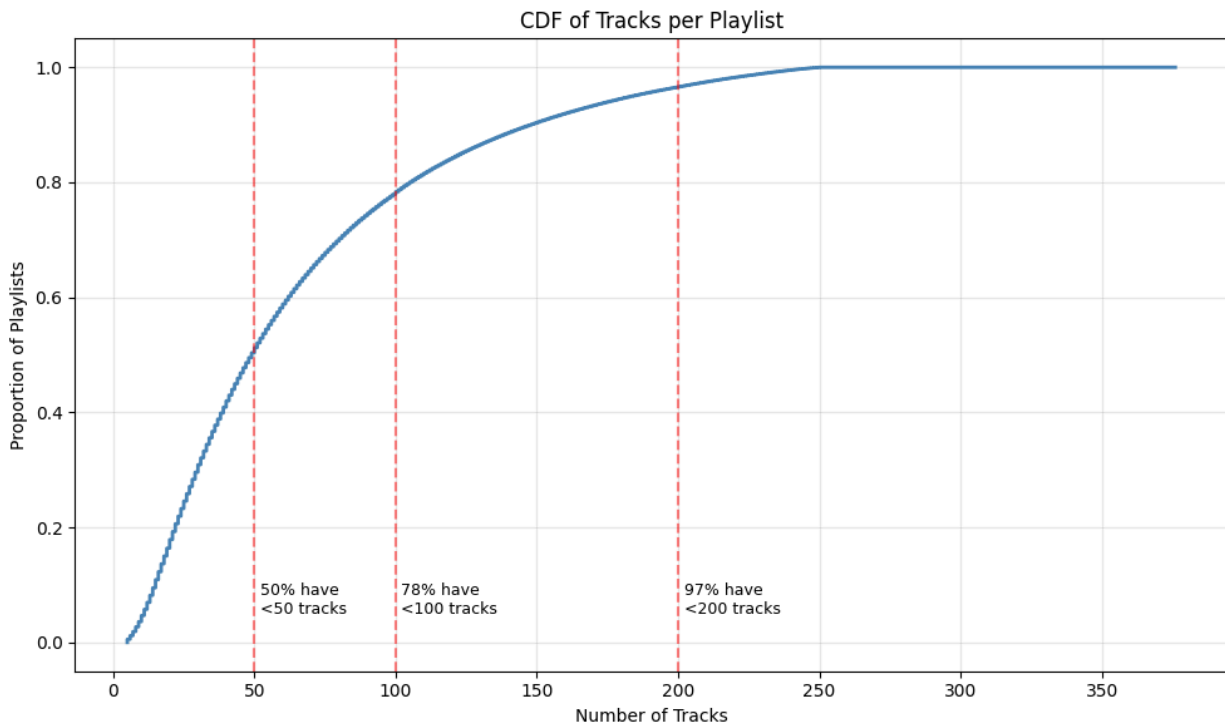
```
import matplotlib.pyplot as plt
import numpy as np

track_counts = sorted([len(tracks[k]) for k in real_keys])
cdf = np.arange(1, len(track_counts) + 1) / len(track_counts)

plt.figure(figsize=(10, 6))
plt.plot(track_counts, cdf, color='steelblue', linewidth=2)
plt.xlabel('Number of Tracks')
plt.ylabel('Proportion of Playlists')
plt.title('CDF of Tracks per Playlist')
plt.grid(True, alpha=0.3)

# Mark useful reference points
for threshold in [50, 100, 200]:
    idx = np.searchsorted(track_counts, threshold)
    pct = cdf[idx] * 100
    plt.axvline(x=threshold, color='red', linestyle='--', alpha=0.5)
    plt.text(threshold + 2, 0.05, f'{pct:.0f}% have \n< {threshold} tracks', fontsize=9)

plt.tight_layout()
plt.show()
```

Box plot of track counts by most common title categories.

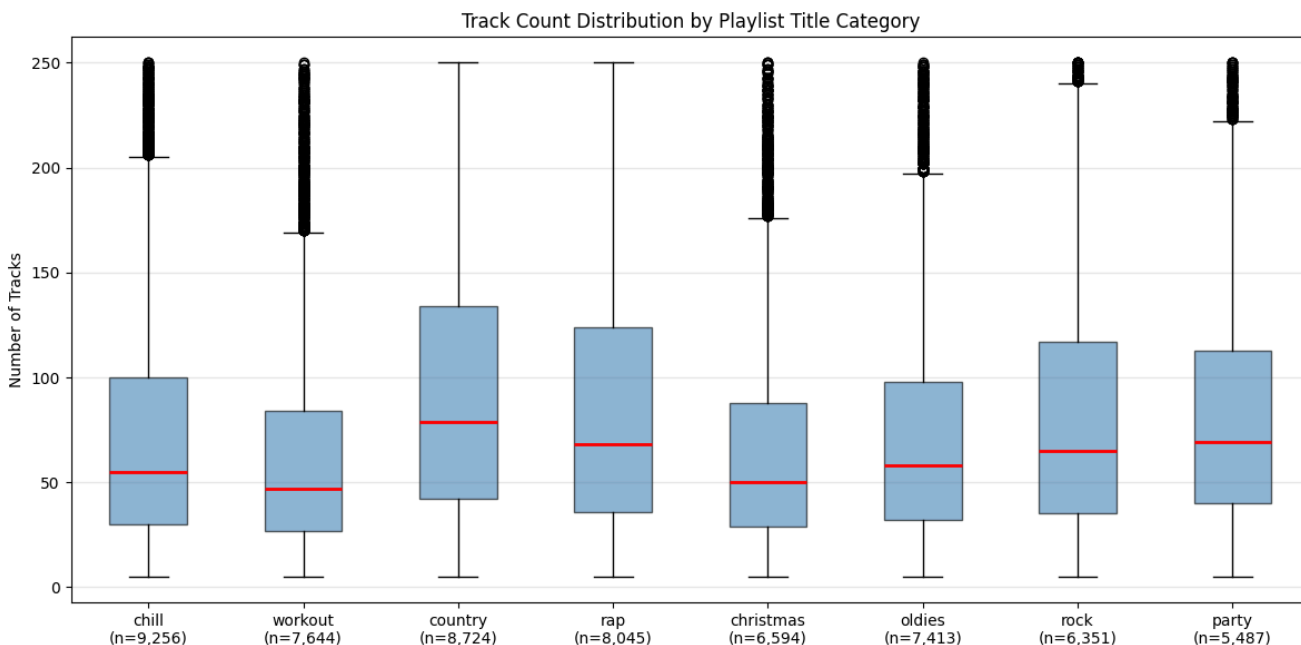
```
import matplotlib.pyplot as plt

# Define the categories to compare
categories = ['chill', 'workout', 'country', 'rap', 'christmas', 'oldies', 'rock', 'party']

# Gather track counts for each category
grouped_data = []
labels = []
for cat in categories:
    counts = [len(tracks[k]) for k in real_keys if titles_lower[k] == cat]
    if counts:
        grouped_data.append(counts)
        labels.append(f"{cat}\n(n={len(counts):,})")

plt.figure(figsize=(12, 6))
plt.boxplot(grouped_data, labels=labels, patch_artist=True,
            boxprops=dict(facecolor='steelblue', alpha=0.6),
            medianprops=dict(color='red', linewidth=2))
plt.ylabel('Number of Tracks')
plt.title('Track Count Distribution by Playlist Title Category')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_22294/568094143.py:16: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' si
plt.boxplot(grouped_data, labels=labels, patch_artist=True,
```



2. Manual Inspection of Example Playlists

Quantitative summaries alone can obscure the qualitative character of the data. To build intuition about what the playlists actually look like, we print a sample of individual entries in full: the title, playlist ID, and the first several track names. This kind of manual inspection is a standard first step in any data analysis project: it helps identify unexpected patterns, data quality issues, or interesting variation that aggregate statistics would miss.

```
# Print the first 5 real playlists with their title and up to 8 tracks each
for k in real_keys[:5]:
    print(f"[ID: {k}] '{titles[k]}' ({len(tracks[k])} tracks)")
    for track in tracks[k][:8]:
        print(f"    - {track}")
    if len(tracks[k]) > 8:
        print(f"    ... and {len(tracks[k]) - 8} more")
    print()
```

```
[ID: 0] 'Throwbacks' (52 tracks)
    - Lose Control (feat. Ciara & Fat Man Scoop)
    - Toxic
    - Crazy In Love
    - Rock Your Body
    - It Wasn't Me
    - Yeah!
    - My Boo
    - Buttons
    ... and 44 more
```

```
[ID: 1] 'Awesome Playlist' (39 tracks)
    - Eye of the Tiger
    - Libera Me From Hell (Tengen Toppa Gurren Lagann)
    - Pokémon Theme
    - Concerning Hobbits (The Lord of the Rings)
    - The Blood of Cuchulainn (The Boondock Saints)
    - He's a Pirate (Pirates of the Caribbean)
    - Very Bloody Tears (Castlevania II: Simon's Quest)
    - U.N. Owen Was Her? (Remix)
    ... and 31 more
```

```
[ID: 2] 'korean ' (64 tracks)
    - Like You
    - GOOD (feat. ELO)
    - Inferiority Complex (feat. Eunha)
    - Ordinary Love
    - Spring Day
    - Ah-Choo
```

```

- BREATHE
- FXXK WIT US
... and 56 more

[ID: 3] 'mat' (126 tracks)
- Danse macabre
- "Piano concerto No. 2 in G Minor
- Drøm Girl
- Sad Valentine
- You Say I'm in Love
- Drowsy
- Serving Goffman
- Sea Song
... and 118 more

[ID: 4] '90s' (17 tracks)
- "Tonight
- Wonderwall - Remastered
- "I Don't Want to Miss a Thing - From the Touchstone film
- Jealousy - Remastered Single Version
- Waterfalls
- 1979
- Torn
- Stay
... and 9 more

```

The information from looking at a couple playlists gives us some interesting information. Playlists are thematically coherent. Each one has a clear organizing principle that a human would immediately recognize: nostalgia (throwbacks), epic/adventure music (Awesome Playlist), K-pop (korean), and decade or era based (90s). This is encouraging for the embeddings, because it means there is real semantic structure for the model to capture. The playlist titles are often informal and short. "mat" (ID 3) may be someone's name or abbreviation, "korean " has a trailing space, and "90s" is purely functional. This is consistent with the median title length of 7 characters, people are not writing descriptive titles, they are writing quick labels for personal use. Playlist length varies enormously. ID 4 has only 17 tracks while ID 3 has 126. This is consistent with the right skewed track count histogram and the statistics (mean 66, median 49, max 376). "Awesome Playlist" (ID 1) is a different type of title and playlist. It contains video game music, anime soundtracks, and movie scores, genres probably would not appear together in a commercial playlist. This suggests the embedding for this playlist might sit in an unusual part of the space, since it is harder to categorize by a single genre. The data quality is imperfect and visual inspection can identify typos and formatting inconsistencies. For example, there are quotation marks at the start of titles like "Tonight and "Piano concerto No. 2, and there is an extra space after "korean ". These typos could introduce noise into the embeddings.

3. Nearest Neighbors in Raw Embedding Space

A central motivation for using embedding representations is that semantically similar playlists should map to nearby points in vector space. To test this, we perform a nearest neighbor search: given a query playlist, we find the playlists whose embeddings are most similar to it under cosine similarity. Cosine similarity is preferred over Euclidean distance for high-dimensional embeddings because it is invariant to the magnitude of the vectors and focuses purely on the direction.

To make the computation tractable, we work with a sample of 5,000 playlists. We first normalize all embeddings to unit length, after which the cosine similarity between any two vectors reduces to their dot product. This allows us to compute similarities for all 5,000 pairs simultaneously using a single matrix multiplication.

```

# Stack a sample of embeddings into a matrix for efficient computation
sample_keys = real_keys[:5000] # use first 5000 playlists
emb_matrix = np.stack([embeddings[k] for k in sample_keys]) # shape: (5000, 768)

# Normalize each embedding to unit length so dot product = cosine similarity
norms = np.linalg.norm(emb_matrix, axis=1, keepdims=True)
emb_matrix_norm = emb_matrix / norms

print(f"Embedding matrix shape: {emb_matrix.shape}")

Embedding matrix shape: (5000, 768)

```

Querying the Nearest Neighbors

We select a query playlist and retrieve its five nearest neighbors by cosine similarity. For each neighbor, we print the playlist title, its similarity score, and a sample of its tracks. If the embeddings are meaningful, we would expect the nearest neighbors to share a similar theme or genre with the query playlist — for example, a workout playlist should retrieve other energetic or upbeat playlists rather than ambient or classical ones.

```

# Select a query playlist by index (change query_idx to explore different queries)
query_idx = 0

```

```

query_vec = emb_matrix_norm[query_idx] # unit-normalized embedding of the query

# Cosine similarity = dot product of unit vectors; compute against all 5000 playlists at once
# the @ symbol is matrix multiplication
# result is one similarity score per playlist with the selected playlist
similarities = emb_matrix_norm @ query_vec # shape: (5000,)

# Rank by similarity; skip index 0 (the query itself) and take the top 5
top_indices = np.argsort(similarities)[::-1][1:6]

print(f"Query playlist: '{titles[sample_keys[query_idx]]}'")
print(f"Sample tracks: {tracks[sample_keys[query_idx]][1:5]}\n")
print("Top 5 nearest neighbors:")
for rank, idx in enumerate(top_indices, 1):
    k = sample_keys[idx]
    print(f" {rank}. '{titles[k]}' (cosine similarity: {similarities[idx]:.4f})")

```

```

Query playlist: 'Throwbacks'
Sample tracks: ['Lose Control (feat. Ciara & Fat Man Scoop)', 'Toxic', 'Crazy In Love', 'Rock Your Body', "It Wasn't Me"]

Top 5 nearest neighbors:
1. 'throwbacks' (cosine similarity: 0.9552)
   Sample tracks: ['My Humps', 'Single Ladies (Put a Ring on It)', 'Hollaback Girl']
2. 'Throwback' (cosine similarity: 0.9552)
   Sample tracks: ['Do It To It - Feat. Sean Paul Of YoungBloodZ', 'Lose My Breath', 'Bills']
3. 'throwbacks' (cosine similarity: 0.9548)
   Sample tracks: ['Girlfriend', 'All You Wanted', 'Hey Ya! - Radio Mix / Club Mix']
4. 'throwback' (cosine similarity: 0.9544)
   Sample tracks: ['Sk8er Boi', "Stacy's Mom", 'Year 3000']
5. 'Throwback Jams' (cosine similarity: 0.9526)
   Sample tracks: ['Heartless', 'I Gotta Feeling', 'What the Hell']

```

The nearest neighbor results show that the mean-pooled embeddings successfully capture the similarity between playlists. Using "Throwbacks" as a query, all five nearest neighbors playlists also had early 2000s music. The cosine similarity scores ranged from 0.9526 to 0.9552, meaning these playlists occupy nearly the same region of the embedded space.

The similarity is driven by track content rather than title text. Case variants of the same title ("Throwbacks", "throwbacks", "Throwback") were retrieved as distinct playlists with nearly identical scores, confirming that the model does not use title information. Looking at the retrieved track lists, some neighbors lean toward early 2000s R&B and pop while others skew toward pop-punk and alternative. The query playlist spans both subgenres, which explains why it sits centrally within this neighborhood. Overall, these results suggest that nearest neighbor search is a viable approach for finding thematically related playlists.